UNIVERSIDAD TECNOLÓGICA DEL PERÚ

Desarrollo Web Integrado · 100000ST61

LABORATORIO GUÍA 9

Autenticación con JWT en una API REST con Spring Boot

Proyecto «Registro de Notas» (pe.utp.notasapp) · IntelliJ IDEA · Maven · JDK 21

Dato	Detalle
Unidad / Semana / Sesión	Unidad 2 · Semana 9 · Sesión 9: Implementación de autenticación con JWT
Carrera	Ingeniería de Software
Modalidad	En parejas (trabajo guiado + autónomo)
Duración estimada	165 minutos (se puede distribuir entre la clase y el trabajo autónomo)
Herramientas	IntelliJ IDEA · JDK 21 · Maven · Spring Boot 4 · Postman · Navegador web
Entregable	Proyecto comprimido (.zip) + documento con las evidencias de la Fase 10

Cómo usar esta guía

Este laboratorio desarrolla, paso a paso, los **tres ejemplos** y los **tres ejercicios** (inicial, intermedio y avanzado) de la sesión. Al final de cada fase hay un **punto de control**: no avances si no se cumple.

1. Objetivo y logro esperado

Logro de la unidad (sílabo): al finalizar la unidad, el estudiante construye aplicaciones RESTful con conexión a bases de datos relacionales mediante el framework Spring Boot, considerando políticas de seguridad.

Logro de este laboratorio: al finalizar, el estudiante implementa autenticación con JWT en una API REST con Spring Boot: genera y valida tokens firmados, configura un filtro stateless y protege endpoints por rol, y comprueba su funcionamiento en IntelliJ IDEA y Postman.

Escenario

La facultad necesita una API para el **Registro de Notas**. Los **docentes** registran notas; los **alumnos** consultan únicamente las suyas. Hoy cualquier persona podría llamar a la API. Tu misión es protegerla: el usuario inicia sesión una vez, recibe un **token JWT** y lo envía en cada petición.

Usuarios de prueba

Usuario	Clave	Rol	¿Qué puede hacer?
docente1	clave123	DOCENTE	Listar y registrar notas
alumno1	clave123	ALUMNO	Ver solo sus notas (mis-notas)
alumno2	clave123	ALUMNO	Ver solo sus notas (mis-notas)

Mapa del laboratorio

Fase	Contenido	Tiempo	Relación con la sesión
0	Verificar JDK 21 en IntelliJ IDEA	5 min	Preparación
1	Crear el proyecto Maven con Spring Boot 4	15 min	Preparación
2	Dependencias JJWT y configuración	10 min	Diapositiva «Dependencias»
3	Dominio: Nota, repositorio y controller	15 min	Proyecto base
4	JwtService: generar y validar el token	15 min	**Ejemplo 1**
5	Prueba JUnit y decodificación de un token	15 min	**Ejercicio 1 (inicial)**
6	Login y configuración básica de seguridad	20 min	**Ejemplo 2**
7	Filtro JWT y reglas por rol	20 min	**Ejemplo 3**
8	Login con validaciones y errores	15 min	**Ejercicio 2 (intermedio)**
9	Blindar el Registro de Notas	25 min	**Ejercicio 3 (avanzado)**
10	Evidencias, entrega y evaluación	10 min	Cierre

Requisitos previos

Herramienta	Versión	Para qué se usa
IntelliJ IDEA	Community o Ultimate (versión reciente)	Editar, ejecutar y probar el proyecto
JDK	21 (Temurin, Oracle OpenJDK u otro)	Compilar y ejecutar Java 21
Maven	Incluido en IntelliJ (no necesitas instalarlo)	Gestionar dependencias y ejecutar pruebas
Spring Boot	4.0.x (última estable)	Framework de la API
JJWT	0.13.0	Crear y validar tokens JWT
Postman	Versión de escritorio o web	Probar los endpoints
Conexión a Internet	—	Descargar dependencias de Maven la primera vez

> **Nota**
> Antes de empezar, revisa que sabes usar lo visto en sesiones previas: controladores REST (Sesión 4), JPA y repositorios (Sesiones 6 y 7), y roles en Spring Security (Sesión 8).

Fase 0 · Verificar JDK 21 en IntelliJ IDEA

Meta: confirmar que IntelliJ usa JDK 21. Spring Boot 4 exige como mínimo Java 17; en este curso trabajamos con Java 21.

Paso 0.1 · Revisar si JDK 21 está instalado

1. Abre una terminal (Windows: `cmd` o PowerShell) y escribe `java -version`.
2. Si ves una línea como `openjdk version "21.x.x"`, ya lo tienes. Si no, continúa con el paso 0.2 (IntelliJ puede descargarlo por ti).

Paso 0.2 · Configurar el JDK dentro de IntelliJ

Esto se hace cuando ya tengas un proyecto abierto (Fase 1). Recuerda la ruta:

1. Menú **File → Project Structure…** (atajo `Ctrl + Alt + Shift + S`).
2. En **Project**, abre la lista **SDK**. Si aparece 21, selecciónalo. Si no, elige **Add SDK → Download JDK…**, escoge **Version: 21**, un proveedor (por ejemplo «Eclipse Temurin») y pulsa **Download**.
3. En **Language level** elige **21**. Pulsa **Apply** y **OK**.
4. Abre **File → Settings → Build, Execution, Deployment → Build Tools → Maven → Runner** y verifica que **JRE** sea «Use Project JDK» (o 21).

> **Punto de control**
> `java -version` muestra 21, o IntelliJ muestra **SDK 21** y **Language level 21** en Project Structure.

Fase 1 · Crear el proyecto Maven con Spring Boot 4

Meta: tener un proyecto `notasapp` que arranque en IntelliJ con Java 21.

Paso 1.1 · Generar el proyecto en Spring Initializr

Funciona tanto en IntelliJ Community como Ultimate: usaremos la web de Spring Initializr.

1. Abre `https://start.spring.io` en el navegador.
2. Completa los campos de la tabla siguiente y agrega las 4 dependencias con el botón **ADD DEPENDENCIES…**.
3. Pulsa **GENERATE**: se descargará `notasapp.zip`. Descomprímelo en una carpeta de trabajo (por ejemplo `C:\proyectos\notasapp`).

Campo	Valor
Project	Maven
Language	Java
Spring Boot	4.0.x (elige la última versión estable: sin SNAPSHOT ni M)
Group	pe.utp
Artifact / Name	notasapp
Package name	pe.utp.notasapp
Packaging	Jar
Java	21
Dependencias	Spring Web · Spring Security · Spring Data JPA · H2 Database

Paso 1.2 · Abrir el proyecto en IntelliJ IDEA

1. En IntelliJ: **File → Open…** y selecciona la carpeta `notasapp` (la que contiene `pom.xml`).
2. Si IntelliJ pregunta, elige **Trust Project** y, si lo ofrece, **Open as Project**.
3. Espera a que termine la barra de progreso inferior (descarga de dependencias). Si no inicia, abre la pestaña **Maven** (derecha) y pulsa el icono **Reload All Maven Projects** (flechas circulares).
4. Aplica la **Fase 0 · Paso 0.2** si IntelliJ no detecta el JDK 21.

Paso 1.3 · Conocer la estructura

```
notasapp
├── pom.xml
└── src
    ├── main
    │   ├── java/pe/utp/notasapp/NotasappApplication.java
    │   └── resources/application.properties
    └── test
        └── java/pe/utp/notasapp/NotasappApplicationTests.java
```

Paso 1.4 · Crear los paquetes del laboratorio

Haz clic derecho sobre el paquete `pe.utp.notasapp` → **New** → **Package** y crea estos cuatro paquetes (uno a uno):

Paquete	Contendrá
pe.utp.notasapp.config	ApplicationConfig, SecurityConfig, DataLoader
pe.utp.notasapp.security	JwtService, JwtAuthenticationFilter
pe.utp.notasapp.auth	AuthController, LoginRequest, LoginResponse
pe.utp.notasapp.nota	Nota, NotaRepository, NotaController

Paso 1.5 · Primera ejecución

1. Abre `NotasappApplication.java` y pulsa el triángulo verde junto a `main` → **Run 'NotasappApplication'**.
2. En la consola debes ver `Started NotasappApplication` y una línea `Using generated security password: …` (Spring Security está activo).
3. Detén la aplicación con el cuadrado rojo **Stop**.

Si algo falla

`release version 21 not supported` / `invalid source release`: el JDK del proyecto no es 21. Repite la Fase 0.

`Port 8080 was already in use`: otra aplicación usa el puerto. Ciérrala o agrega `server.port=8081` en `application.properties`.

Dependencias en rojo: pulsa **Reload All Maven Projects** y verifica tu conexión a Internet.

Punto de control

La aplicación arranca sin errores y muestra `Started NotasappApplication`.

Fase 2 · Dependencias JJWT y configuración

Meta: agregar la librería JJWT y definir la clave secreta y la vigencia del token.

Paso 2.1 · Agregar JJWT al pom.xml

Abre `pom.xml` y, dentro de `<dependencies>`, agrega estas tres dependencias (misma versión):

Archivo: `pom.xml`

```
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-api</artifactId>
    <version>0.13.0</version>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-impl</artifactId>
    <version>0.13.0</version>
    <scope>runtime</scope>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-jackson</artifactId>
    <version>0.13.0</version>
    <scope>runtime</scope>
</dependency>
```

Pulsa el icono **Load Maven Changes** que aparece en el borde del editor (o **Reload All Maven Projects**).

> **Nota**
>
> `0.13.0` era la última versión al preparar esta guía. Puedes consultar si hay una más reciente en Maven Central; las tres librerías deben usar **la misma versión**.

Paso 2.2 · Configurar application.properties

Archivo: `src/main/resources/application.properties`

```
spring.application.name=notasapp

# Clave secreta en Base64 (minimo 32 bytes). En produccion: variable JWT_SECRET
jwt.secret=${JWT_SECRET:dXRwLXNlc2lvbjktand0LWNsYXZlLWRlbW8tMjU2Yml0cy0yMDI2}

# Vigencia del token en milisegundos (3600000 = 1 hora)
jwt.expiration-ms=3600000
```

> **Importante**
>
> Esta clave es **solo para el laboratorio**. En un proyecto real se define con la variable de entorno `JWT_SECRET` y **nunca** se sube al repositorio.
>
> Si la clave tiene menos de 256 bits, JJWT lanza `WeakKeyException` al arrancar.

> **Punto de control**
>
> Maven recarga sin errores y el proyecto sigue arrancando (ejecútalo otra vez si quieres confirmarlo).

Fase 3 · Dominio: Nota, repositorio y controller

Meta: crear la parte funcional del Registro de Notas (JPA + H2) que más adelante protegeremos.

Paso 3.1 · Entidad Nota

Clic derecho en el paquete `nota` → **New** → **Java Class** → `Nota`.

Archivo: `src/main/java/pe/utp/notasapp/nota/Nota.java`

```
package pe.utp.notasapp.nota;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

@Entity
public class Nota {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String alumnoUsername;
    private String curso;
    private double valor;

    public Nota() {
    }

    public Nota(String alumnoUsername, String curso, double valor) {
        this.alumnoUsername = alumnoUsername;
        this.curso = curso;
        this.valor = valor;
    }

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getAlumnoUsername() { return alumnoUsername; }
    public void setAlumnoUsername(String alumnoUsername) {
        this.alumnoUsername = alumnoUsername;
    }

    public String getCurso() { return curso; }
    public void setCurso(String curso) { this.curso = curso; }

    public double getValor() { return valor; }
    public void setValor(double valor) { this.valor = valor; }
}
```

Paso 3.2 · Repositorio

Archivo: src/main/java/pe/utp/notasapp/nota/NotaRepository.java

```
package pe.utp.notasapp.nota;

import org.springframework.data.jpa.repository.JpaRepository;

import java.util.List;

public interface NotaRepository extends JpaRepository<Nota, Long> {

    List<Nota> findByAlumnoUsername(String alumnoUsername);
}
```

Paso 3.3 · Controller base

Archivo: src/main/java/pe/utp/notasapp/nota/NotaController.java

```
package pe.utp.notasapp.nota;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

@RestController
@RequestMapping("/api/notas")
public class NotaController {

    private final NotaRepository repo;

    public NotaController(NotaRepository repo) {
        this.repo = repo;
    }

    @GetMapping
    public List<Nota> listar() {
        return repo.findAll();
    }

    @PostMapping
    public Nota registrar(@RequestBody Nota nota) {
        return repo.save(nota);
    }
}
```

Paso 3.4 · Datos de prueba

Archivo: `src/main/java/pe/utp/notasapp/config/DataLoader.java`

```
package pe.utp.notasapp.config;

import org.springframework.boot.CommandLineRunner;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import pe.utp.notasapp.nota.Nota;
import pe.utp.notasapp.nota.NotaRepository;

@Configuration
public class DataLoader {

    @Bean
    CommandLineRunner cargarNotas(NotaRepository repo) {
        return args -> {
            repo.save(new Nota("alumno1", "Desarrollo Web Integrado", 17.0));
            repo.save(new Nota("alumno1", "Base de Datos", 15.5));
            repo.save(new Nota("alumno2", "Desarrollo Web Integrado", 14.0));
        };
    }
}
```

Nota

Aún no probamos los endpoints en Postman: Spring Security ya está activo y los bloquea. Eso es justo lo que arreglaremos en las siguientes fases.

Punto de control

La aplicación arranca sin errores. En la consola aparecen líneas de Hibernate/H2 y `Started NotasappApplication`.

Si algo falla

`Cannot resolve symbol 'jakarta'`: recarga Maven; la dependencia `spring-boot-starter-data-jpa` debe estar en `pom.xml`.

Imports sin resolver: coloca el cursor sobre el texto en rojo y pulsa `Alt + Enter` → «Import class». Para ordenar imports: `Ctrl + Alt + O`.

Fase 4 · Ejemplo 1: JwtService (generar y validar)

Meta: crear el servicio que emite y verifica tokens firmados con HS256.

Paso 4.1 · Crear la clase JwtService

Archivo: `src/main/java/pe/utp/notasapp/security/JwtService.java`

```
package pe.utp.notasapp.security;

import io.jsonwebtoken.Claims;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.io.Decoders;
import io.jsonwebtoken.security.Keys;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.stereotype.Service;

import javax.crypto.SecretKey;
import java.util.Date;

@Service
public class JwtService {

    private final SecretKey key;
    private final long expirationMs;

    public JwtService(@Value("${jwt.secret}") String secret,
                      @Value("${jwt.expiration-ms}") long expirationMs) {
        this.key = Keys.hmacShaKeyFor(Decoders.BASE64.decode(secret));
        this.expirationMs = expirationMs;
    }

    public long getExpirationMs() {
        return expirationMs;
    }

    /** Genera un token firmado con usuario, rol y vencimiento. */
    public String generarToken(UserDetails user) {
        Date ahora = new Date();
        String rol = user.getAuthorities().stream()
                .findFirst().get().getAuthority();
        return Jwts.builder()
                .subject(user.getUsername())
                .claim("rol", rol)
                .issuedAt(ahora)
                .expiration(new Date(ahora.getTime() + expirationMs))
                .signWith(key)
                .compact();
    }

    /** Verifica firma y vencimiento; lanza JwtException si algo falla. */
    public Claims extraerClaims(String token) {
        return Jwts.parser()
                .verifyWith(key)
                .build()
                .parseSignedClaims(token)
                .getPayload();
    }

    public String extraerUsername(String token) {
        return extraerClaims(token).getSubject();
    }
}
```

Paso 4.2 · Entender el código

Línea	¿Qué hace?
`Keys.hmacShaKeyFor(...)`	Convierte la clave Base64 en una `SecretKey` apta para HMAC.
`.subject(...)` y `.claim("rol", ...)`	Escriben usuario y rol en el payload.
`.expiration(...)`	Define el vencimiento: ahora + vigencia configurada.
`.signWith(key)`	Firma el token; elige HS256 según el tamaño de la clave.
`.compact()`	Arma el texto `header.payload.signature`.
`parseSignedClaims(token)`	Verifica la firma y el vencimiento; si falla, lanza una excepción (`JwtException`).

Punto de control

El proyecto compila sin errores (menú **Build → Build Project**, atajo `Ctrl + F9`).

Si algo falla

`WeakKeyException`: la clave de `application.properties` es muy corta. Copia exactamente la de la Fase 2.

`Could not resolve placeholder 'jwt.secret'`: revisa que el nombre de la propiedad esté bien escrito.

Si en un tutorial ves `Jwts.parserBuilder()` o `signWith(key, SignatureAlgorithm.HS256)`, son formas **obsoletas** de versiones antiguas de JJWT.

Fase 5 · Ejercicio 1 (inicial): decodificar y probar un token

Meta: interpretar un JWT y escribir una prueba unitaria con JUnit que valide `JwtService`.

Paso 5.1 · Parte A: descifrar el token

El siguiente token fue firmado con la clave del laboratorio:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJkb2NlbnRlMSIsInJvbCI6IlJPTEVfRE9DRU5URSIsImlhdCI6MTc4ODAwMDAwMCwiZXhwIjoxNzg4MDAzNjAwfQ.fRQl0HWN0IBHC1V4aAqcK0GSk1MxSbYkRevRpHB6vDc
```

Rojo = header · Violeta = payload · Celeste = firma.

1. Abre un decodificador de JWT en el navegador (por ejemplo `jwt.io`) y pega el token.
2. Completa la tabla de respuestas.

Pregunta	Tu respuesta
¿Qué algoritmo de firma usa?	
¿Quién es el usuario (`sub`) y cuál es su rol?	
¿Cuándo se emitió (`iat`) y cuándo vence (`exp`)?	
¿Ya venció hoy?	
¿Pudiste leer el payload sin conocer la clave? ¿Por qué?	

> **Importante**
> **Nunca** pegues tokens reales de producción en sitios web de terceros. Aquí es seguro porque es un token de práctica.

Paso 5.2 · Parte B: crear la prueba JUnit

1. Clic derecho en `src/test/java` → paquete `pe.utp.notasapp.security` (créalo si no existe) → **New → Java Class** → `JwtServiceTest`.
2. Escribe el código siguiente. Observa que `TOKEN_VENCIDO` es el token de la Parte A, dividido en tres líneas (header, payload, firma).

Archivo: `src/test/java/pe/utp/notasapp/security/JwtServiceTest.java`

```
package pe.utp.notasapp.security;

import io.jsonwebtoken.ExpiredJwtException;
import org.junit.jupiter.api.Test;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertThrows;

class JwtServiceTest {

    static final String SECRET =
            "dXRwLXNlc2lvbjktand0LWNsYXZlLWRlbW8tMjU2Yml0cy0yMDI2";
    static final String TOKEN_VENCIDO =
            "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9."
          + "eyJzdWIiOiJkb2NlbnRlMSIsInJvbCI6IlJPTEVfRE9DRU5URSIsImlh"
          + "dCI6MTc4ODAwMDAwMCwiZXhwIjoxNzg4MDAzNjAwfQ."
```

```
            + "fRQl0HWN0IBHC1V4aAqcK0GSk1MxSbYkRevRpHB6vDc";

    final JwtService jwt = new JwtService(SECRET, 3_600_000);

    @Test
    void generaTokenValido() {
        UserDetails u = User.withUsername("alumno1")
                .password("x").roles("ALUMNO").build();

        String token = jwt.generarToken(u);

        assertEquals(3, token.split("\\.").length);
        assertEquals("alumno1", jwt.extraerUsername(token));
        assertEquals("ROLE_ALUMNO",
                jwt.extraerClaims(token).get("rol", String.class));
    }

    @Test
    void tokenVencidoLanzaExcepcion() {
        assertThrows(ExpiredJwtException.class,
                () -> jwt.extraerClaims(TOKEN_VENCIDO));
    }
}
```

Paso 5.3 · Ejecutar la prueba

1. Pulsa el icono verde junto a `class JwtServiceTest` → **Run 'JwtServiceTest'**.
2. En la ventana **Run** deben aparecer las dos pruebas en verde.
3. Opcional: en la pestaña **Maven** → «notasapp → Lifecycle» → doble clic en **test** para ejecutar todas las pruebas con Maven.

Solución de la Parte A

Pregunta	Respuesta
Algoritmo	HS256 (HMAC con SHA-256)
Usuario y rol	`docente1` · `ROLE_DOCENTE`
Emisión y vencimiento	Emitido el 29/08/2026 (10:40 UTC); vence 1 hora después
¿Ya venció?	Sí. Por eso `extraerClaims` lanza `ExpiredJwtException`
¿Se leyó sin la clave?	Sí: el payload solo está codificado (Base64URL), no cifrado. La clave solo sirve para **verificar la firma**.

Punto de control

`JwtServiceTest` pasa con **2 pruebas en verde**.

Si algo falla

`cannot find symbol: JwtService`: el paquete de la prueba debe ser `pe.utp.notasapp.security`, igual que la clase.

La prueba del token vencido falla con `SignatureException`: el `SECRET` de la prueba no coincide con el de la Fase 2, o copiaste mal alguna parte del token.

`Cannot resolve symbol 'junit'`: recarga Maven; el proyecto debe incluir `spring-boot-starter-test` o los starters de prueba generados por Initializr.

Fase 6 · Ejemplo 2: login y configuración básica

Meta: crear el endpoint `POST /api/auth/login` que devuelve un token, y una primera configuración de seguridad **sin** filtro (para ver por qué lo necesitamos).

Paso 6.1 · Beans de apoyo

Archivo: `src/main/java/pe/utp/notasapp/config/ApplicationConfig.java`

```
package pe.utp.notasapp.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.authentication.AuthenticationManager;
import
org.springframework.security.config.annotation.authentication.configuration.Authenticat
ionConfiguration;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;

@Configuration
public class ApplicationConfig {

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public UserDetailsService userDetailsService(PasswordEncoder enc) {
        UserDetails docente = User.withUsername("docente1")
                .password(enc.encode("clave123")).roles("DOCENTE").build();
        UserDetails alumno1 = User.withUsername("alumno1")
                .password(enc.encode("clave123")).roles("ALUMNO").build();
        UserDetails alumno2 = User.withUsername("alumno2")
                .password(enc.encode("clave123")).roles("ALUMNO").build();
        return new InMemoryUserDetailsManager(docente, alumno1, alumno2);
    }

    @Bean
    public AuthenticationManager authenticationManager(
            AuthenticationConfiguration config) throws Exception {
        return config.getAuthenticationManager();
    }
}
```

> **Nota**
>
> Usamos usuarios en memoria para simplificar. En tu proyecto final los reemplazarás por un `UserDetailsService` que consulte la base de datos con JPA.

Paso 6.2 · Records de entrada y salida del login

Archivo: `src/main/java/pe/utp/notasapp/auth/LoginRequest.java`

```
package pe.utp.notasapp.auth;

public record LoginRequest(String username, String password) {
}
```

Archivo: src/main/java/pe/utp/notasapp/auth/LoginResponse.java

```
package pe.utp.notasapp.auth;

public record LoginResponse(String token, String tipo) {
}
```

Paso 6.3 · AuthController (versión básica)

Archivo: src/main/java/pe/utp/notasapp/auth/AuthController.java

```
package pe.utp.notasapp.auth;

import org.springframework.http.ResponseEntity;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import pe.utp.notasapp.security.JwtService;

@RestController
@RequestMapping("/api/auth")
public class AuthController {

    private final AuthenticationManager authManager;
    private final JwtService jwtService;

    public AuthController(AuthenticationManager authManager,
                          JwtService jwtService) {
        this.authManager = authManager;
        this.jwtService = jwtService;
    }

    @PostMapping("/login")
    public ResponseEntity<LoginResponse> login(
            @RequestBody LoginRequest req) {

        Authentication auth = authManager.authenticate(
                new UsernamePasswordAuthenticationToken(
                        req.username(), req.password()));

        UserDetails user = (UserDetails) auth.getPrincipal();
        String token = jwtService.generarToken(user);
        return ResponseEntity.ok(new LoginResponse(token, "Bearer"));
    }
}
```

Paso 6.4 · SecurityConfig (versión 1, sin filtro)

Archivo: `src/main/java/pe/utp/notasapp/config/SecurityConfig.java`

```
package pe.utp.notasapp.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpStatus;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import
org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.HttpStatusEntryPoint;

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http)
            throws Exception {
        http
            .csrf(csrf -> csrf.disable())
            .sessionManagement(s -> s.sessionCreationPolicy(
                    SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**").permitAll()
                .anyRequest().authenticated())
            .exceptionHandling(e -> e.authenticationEntryPoint(
                    new HttpStatusEntryPoint(HttpStatus.UNAUTHORIZED)));
        return http.build();
    }
}
```

Paso 6.5 · Probar en Postman

Ejecuta la aplicación (**Run 'NotasappApplication'**) y prepara Postman:

1. Crea un **Environment** llamado `NotasApp` con la variable `baseUrl` = `http://localhost:8080` y selecciónalo (esquina superior derecha).
2. Crea una **Collection** llamada `NotasApp JWT`.
3. Nueva petición **POST** `{{baseUrl}}/api/auth/login`. En **Body → raw → JSON** (no uses «Params») escribe el JSON de abajo y pulsa **Send**.

```
{
  "username": "alumno1",
  "password": "clave123"
}
```

Respuesta esperada: **200 OK** con un JSON similar a:

```
{
  "token": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOi...",
  "tipo": "Bearer"
}
```

4. En la pestaña **Scripts → Post-response** (o **Tests**, según tu versión de Postman) de esa petición, pega esta línea para guardar el token automáticamente:

```
pm.environment.set("token", pm.response.json().token);
```

5. Crea la petición **GET** `{{baseUrl}}/api/notas`. En la pestaña **Authorization** elige **Type: Bearer Token** y escribe `{{token}}` (sin la palabra «Bearer»). Pulsa **Send**.

Para pensar

Respuesta obtenida: **401 Unauthorized**, aunque enviaste un token válido. ¿Por qué? Porque Spring Security todavía **no sabe leer el token**: falta el filtro. Lo creamos en la siguiente fase.

Punto de control

`POST /api/auth/login` devuelve 200 con un token. `GET /api/notas` con ese token devuelve 401 (esperado por ahora).

Si algo falla

El login devuelve 403 o 401: revisa usuario y clave (`alumno1` / `clave123`) y que el Body esté en «raw → JSON».

`No qualifying bean of type 'UserDetailsService'`: falta `ApplicationConfig` o está fuera del paquete `pe.utp.notasapp` (la clase principal debe estar en el paquete raíz).

Error 404 en `/`: es normal en una API REST pura; solo existen los endpoints `/api/...`.

Fase 7 · Ejemplo 3: filtro JWT y reglas por rol

Meta: crear el filtro que lee el header `Authorization: Bearer …`, valida el token y deja al usuario autenticado en el `SecurityContext`.

Paso 7.1 · Crear JwtAuthenticationFilter

Archivo: `src/main/java/pe/utp/notasapp/security/JwtAuthenticationFilter.java`

```
package pe.utp.notasapp.security;

import io.jsonwebtoken.JwtException;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;

@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    private final JwtService jwt;
    private final UserDetailsService uds;

    public JwtAuthenticationFilter(JwtService jwt, UserDetailsService uds) {
        this.jwt = jwt;
        this.uds = uds;
    }

    @Override
    protected void doFilterInternal(HttpServletRequest req,
            HttpServletResponse res, FilterChain chain)
            throws ServletException, IOException {

        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            try {
                String token = header.substring(7);
                String username = jwt.extraerUsername(token);
                UserDetails ud = uds.loadUserByUsername(username);
                var auth = new UsernamePasswordAuthenticationToken(
                        ud, null, ud.getAuthorities());
                SecurityContextHolder.getContext().setAuthentication(auth);
            } catch (JwtException | IllegalArgumentException
                     | UsernameNotFoundException e) {
                // token invalido o vencido: no se autentica
            }
        }
        chain.doFilter(req, res);
    }
}
```

Paso del filtro	Explicación
1. Leer el header	Busca `Authorization: Bearer <token>`. Si no existe, deja pasar la petición **sin autenticar**.

Paso del filtro	Explicación
2. `substring(7)`	Quita `Bearer ` (7 caracteres) y deja solo el token.
3. `extraerUsername`	Verifica firma y vencimiento y obtiene el usuario (`sub`).
4. `Authentication`	Crea el objeto con el usuario y sus roles.
5. `SecurityContext`	Guarda al usuario como autenticado para esta petición.
6. `chain.doFilter`	Siempre continúa la cadena de filtros.

Paso 7.2 · SecurityConfig (versión 2: filtro + reglas)

Reemplaza **todo el contenido** de `SecurityConfig.java` por esta versión:

Archivo: `src/main/java/pe/utp/notasapp/config/SecurityConfig.java`

```
package pe.utp.notasapp.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.http.HttpStatus;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import
org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.HttpStatusEntryPoint;
import
org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
import pe.utp.notasapp.security.JwtAuthenticationFilter;

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http,
            JwtAuthenticationFilter jwtFilter) throws Exception {
        http
            .csrf(csrf -> csrf.disable())
            .sessionManagement(s -> s.sessionCreationPolicy(
                    SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**").permitAll()
                .requestMatchers(HttpMethod.GET, "/api/notas/**")
                    .hasAnyRole("ALUMNO", "DOCENTE")
                .requestMatchers("/api/notas/**").hasRole("DOCENTE")
                .anyRequest().authenticated())
            .exceptionHandling(e -> e.authenticationEntryPoint(
                    new HttpStatusEntryPoint(HttpStatus.UNAUTHORIZED)))
            .addFilterBefore(jwtFilter,
                    UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }
}
```

> **Importante**
>
> **El orden de las reglas importa:** Spring usa la primera que coincide. Siempre van primero las más específicas y al final `anyRequest()`.
>
> El filtro se inyecta como **parámetro del método** para evitar una dependencia circular.

Paso 7.3 · Probar el flujo completo

Reinicia la aplicación (botón **Rerun**, flechas verdes) y prueba en Postman:

N°	Petición	Esperado
1	POST `/api/auth/login` con `alumno1` (guarda el token)	200 + token
2	GET `/api/notas` **sin** Authorization	401
3	GET `/api/notas` con Bearer Token de `alumno1`	200 + lista de 3 notas
4	POST `/api/notas` con token de `alumno1`	403
5	Login con `docente1` y POST `/api/notas` con su token	200 + nota guardada
6	GET con un token al que le cambiaste un carácter	401

Cuerpo para el POST de notas:

```
{
  "alumnoUsername": "alumno2",
  "curso": "Base de Datos",
  "valor": 16.5
}
```

Para pensar

¿Por qué en la prueba 4 el resultado es **403** y no 401? Porque el alumno **sí está autenticado** (el servidor sabe quién es) pero **no tiene permiso**.

Punto de control

Las 6 pruebas dan el resultado esperado.

Si algo falla

Todo devuelve 401 aunque envíes el token: revisa que `JwtAuthenticationFilter` tenga `@Component`, que el header sea `Authorization` y que en Postman escribas solo `{{token}}` (sin la palabra «Bearer»).

`UnsatisfiedDependencyException` / dependencia circular: no declares el `UserDetailsService` dentro de `SecurityConfig`; debe estar en `ApplicationConfig`.

El token ya venció: vuelve a hacer login para obtener uno nuevo.

Fase 8 · Ejercicio 2 (intermedio): login con errores

Meta: mejorar el login para que valide los campos, responda con un cuerpo más útil y devuelva códigos HTTP correctos.

Requisitos

- Respuesta exitosa con `token`, `tipo`, `rol` y `expiraEnSegundos`.
- Usuario o clave vacíos → **400** con `{ "error": "Usuario y clave son obligatorios" }`.
- Credenciales incorrectas → **401** con `{ "error": "Credenciales inválidas" }`.
- Pruébalo en Postman con 3 casos: correcto, clave errónea y body vacío (`{}`).

> **Intenta primero**
>
> **Pista:** captura `BadCredentialsException` (paquete `org.springframework.security.authentication`) y construye la respuesta con `ResponseEntity.status(...).body(Map.of(...))`.

Solución paso a paso

Paso 8.1 · Amplía el record de respuesta:

Archivo: `src/main/java/pe/utp/notasapp/auth/LoginResponse.java`

```
package pe.utp.notasapp.auth;

public record LoginResponse(String token, String tipo,
                            String rol, long expiraEnSegundos) {
}
```

Paso 8.2 · Reemplaza el contenido de `AuthController.java`:

Archivo: `src/main/java/pe/utp/notasapp/auth/AuthController.java`

```
package pe.utp.notasapp.auth;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.BadCredentialsException;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import pe.utp.notasapp.security.JwtService;

import java.util.Map;

@RestController
@RequestMapping("/api/auth")
public class AuthController {

    private final AuthenticationManager authManager;
    private final JwtService jwtService;

    public AuthController(AuthenticationManager authManager,
                          JwtService jwtService) {
        this.authManager = authManager;
```

```
        this.jwtService = jwtService;
    }

    @PostMapping("/login")
    public ResponseEntity<?> login(@RequestBody LoginRequest req) {

        if (req.username() == null || req.username().isBlank()
                || req.password() == null || req.password().isBlank()) {
            return ResponseEntity.badRequest().body(
                    Map.of("error", "Usuario y clave son obligatorios"));
        }
        try {
            Authentication auth = authManager.authenticate(
                    new UsernamePasswordAuthenticationToken(
                            req.username(), req.password()));
            UserDetails user = (UserDetails) auth.getPrincipal();
            String token = jwtService.generarToken(user);
            String rol = user.getAuthorities().stream()
                    .findFirst().get().getAuthority();
            return ResponseEntity.ok(new LoginResponse(token, "Bearer",
                    rol, jwtService.getExpirationMs() / 1000));
        } catch (BadCredentialsException e) {
            return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(
                    Map.of("error", "Credenciales inválidas"));
        }
    }
}
```

Decisión	Motivo
Validar primero	Evita llamar a `authenticate()` con datos vacíos y responde 400.
`try / catch`	`BadCredentialsException` se convierte en un 401 con mensaje claro (sin revelar si falló el usuario o la clave).
`ResponseEntity<?>`	Permite devolver cuerpos distintos según el caso.
Record `LoginResponse`	Es un DTO tipado; en un proyecto real es mejor que `Map.of` para respuestas exitosas.

Pruebas

Caso	Body	Esperado
Correcto	`{"username":"alumno1","password":"clave123"}`	200 con token, tipo, rol y expiraEnSegundos (3600)
Clave errónea	`{"username":"alumno1","password":"mala"}`	401 `{"error":"Credenciales inválidas"}`
Body vacío	`{}`	400 `{"error":"Usuario y clave son obligatorios"}`

Punto de control

Los 3 casos devuelven el código y el mensaje esperados.

Fase 9 · Ejercicio 3 (avanzado): blindar el Registro de Notas

Meta: cada alumno ve solo sus notas, solo el docente registra notas y cualquier problema con el token responde 401 en formato JSON.

Requisitos

- **ALUMNO:** `GET /api/notas/mis-notas` devuelve únicamente sus notas.
- **DOCENTE:** `POST /api/notas` registra una nota; `GET /api/notas` lista todas.
- **Token ausente, inválido o vencido:** 401 con `{ "error": ... }`.
- **Prueba del vencimiento:** configura `jwt.expiration-ms=60000` y espera 1 minuto.

Endpoint	ALUMNO	DOCENTE	Sin token
GET /api/notas/mis-notas	200	403	401
POST /api/notas	403	200	401
GET /api/notas	403	200	401

Solución paso a paso

Paso 9.1 · Agrega el método `mis-notas` en `NotaController` (junto a `listar` y `registrar`). Necesitas estos imports nuevos: `org.springframework.security.core.Authentication`.

Archivo: `src/main/java/pe/utp/notasapp/nota/NotaController.java` (método nuevo)

```
    @GetMapping("/mis-notas")
    public List<Nota> misNotas(Authentication auth) {
        // el usuario sale del token ya validado por el filtro
        return repo.findByAlumnoUsername(auth.getName());
    }
```

> **Nota**
> El nombre del usuario se obtiene del objeto `Authentication` que dejó el filtro, **no** de un parámetro de la URL. Así un alumno no puede pedir las notas de otro cambiando un dato de la petición.

Paso 9.2 · En `SecurityConfig.java` reemplaza el bloque `authorizeHttpRequests` y el `exceptionHandling` por los siguientes (el resto queda igual). Agrega el import `jakarta.servlet.http.HttpServletResponse` y quita los imports que ya no uses (`HttpMethod`, `HttpStatus`, `HttpStatusEntryPoint`).

Archivo: `src/main/java/pe/utp/notasapp/config/SecurityConfig.java` (bloque modificado)

```
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**").permitAll()
                .requestMatchers(HttpMethod.GET, "/api/notas/mis-notas")
                    .hasRole("ALUMNO")
                .requestMatchers("/api/notas/**").hasRole("DOCENTE")
                .anyRequest().authenticated())
            .exceptionHandling(e -> e.authenticationEntryPoint(
                (req, res, ex) -> {
                    res.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
                    res.setContentType("application/json;charset=UTF-8");
                    res.getWriter().write(
                        "{\"error\":\"Token ausente, invalido o vencido\"}");
                }))
```

> **Importante**
>
> Deja el import de `HttpMethod`: la regla de `mis-notas` lo sigue usando. Solo se eliminan `HttpStatus` y `HttpStatusEntryPoint`.
>
> La regla de `mis-notas` va **antes** de `/api/notas/**`; si las inviertes, la regla general la taparía.

Paso 9.3 · Prueba el vencimiento del token:

1. En `application.properties` cambia `jwt.expiration-ms=60000` (1 minuto) y reinicia la aplicación.
2. Haz login y guarda el token. Ejecuta `GET /api/notas/mis-notas` inmediatamente: **200**.
3. Espera 61 segundos y repite la misma petición con el mismo token: **401** con el JSON de error.
4. **Vuelve a dejar** `jwt.expiration-ms=3600000` al terminar.

Matriz de pruebas finales (Postman)

N°	Prueba	Esperado	Resultado
1	Login `docente1` / `clave123`	200 + token (rol ROLE_DOCENTE)	
2	Login `alumno1` / `clave123`	200 + token (rol ROLE_ALUMNO)	
3	Login con clave errónea	401 `Credenciales inválidas`	
4	Login con body `{}`	400 `Usuario y clave son obligatorios`	
5	GET `/api/notas` sin token	401 con JSON de error	
6	GET `/api/notas` con token de `alumno1`	403	
7	GET `/api/notas` con token de `docente1`	200 + 3 notas	
8	GET `/api/notas/mis-notas` con `alumno1`	200 + solo 2 notas de alumno1	
9	GET `/api/notas/mis-notas` con `alumno2`	200 + solo 1 nota de alumno2	
10	GET `/api/notas/mis-notas` con `docente1`	403	
11	POST `/api/notas` con `docente1`	200 + nota guardada	
12	POST `/api/notas` con `alumno1`	403	
13	GET con token alterado (cambia un carácter)	401	
14	GET con token vencido (vigencia 60 s)	401	

> **Punto de control**
>
> Las 14 pruebas dan el resultado esperado y tienes la captura de cada una.

Fase 10 · Evidencias, entrega y evaluación

Paso 10.1 · Evidencias a entregar

Código	Evidencia	Fase
E1	Captura de IntelliJ con **Project Structure** (SDK 21) y la consola con `Started NotasappApplication`	0 y 1
E2	Captura del `pom.xml` con las tres dependencias JJWT	2
E3	Captura de las 2 pruebas de `JwtServiceTest` en verde y la tabla de la Parte A completa	5
E4	Captura del login exitoso en Postman (200 + token)	6 y 8
E5	Captura de `GET /api/notas` con token → 401 antes del filtro y 200 después	6 y 7
E6	Capturas de los casos 3 y 4 del login (401 y 400)	8
E7	Matriz final de 14 pruebas completada, con capturas de los casos 5, 6, 8, 12, 13 y 14	9
E8	Carpeta del proyecto en `.zip` (sin la carpeta `target`)	Todas

Paso 10.2 · Comprimir el proyecto

1. Cierra IntelliJ o detén la aplicación.
2. Entra a la carpeta `notasapp`, elimina la carpeta `target` si existe y comprime la carpeta completa en `Lab9_JWT_Apellidos.zip`.

Solución de problemas frecuentes

Síntoma	Causa probable	Solución
`release version 21 not supported`	El proyecto usa un JDK menor a 21	Fase 0: Project Structure → SDK 21 y Language level 21
Dependencias en rojo	Maven no recargó	Pestaña Maven → Reload All Maven Projects
`WeakKeyException`	Clave menor a 256 bits	Usa la clave Base64 de la Fase 2 (mínimo 32 bytes)
`Could not resolve placeholder 'jwt.secret'`	Propiedad mal escrita o archivo equivocado	Revisa `application.properties` en `src/main/resources`
Login devuelve 401 con datos correctos	Body en la pestaña Params o JSON mal formado	Body → raw → JSON
Con token válido sigue 401	Falta el filtro, `@Component` o el prefijo Bearer	Revisa Fase 7 y escribe solo `{{token}}` en Authorization
Siempre 403 para el docente	Rol mal escrito en la regla	Usa `hasRole("DOCENTE")`, sin el prefijo `ROLE_`
Dependencia circular al arrancar	`UserDetailsService` dentro de `SecurityConfig`	Déjalo en `ApplicationConfig`

Síntoma	Causa probable	Solución
404 en `http://localhost:8080/`	Es una API sin página inicial	Prueba los endpoints `/api/...`
`Port 8080 was already in use`	Otra aplicación usa el puerto	Detén la ejecución anterior o cambia `server.port`

Retos opcionales

- Agrega un rol **ADMIN** con un `DELETE /api/notas/{id}` permitido solo a ese rol.
- Crea `GET /api/auth/me` que devuelva el usuario y el rol del token.
- Construye las autoridades usando el claim `rol` del token, **sin** consultar el `UserDetailsService` (autenticación stateless pura).
- Implementa un **refresh token**: access token de 15 minutos y un token largo para pedir uno nuevo.
- Mueve los usuarios a la base de datos con JPA y verifica las claves con BCrypt.

Preguntas de evaluación

1. ¿Qué diferencia hay entre una sesión en el servidor y un token JWT? ¿Qué significa que la API sea «stateless»?
2. Nombra las tres partes de un JWT y explica para qué sirve cada una.
3. Un compañero dice: «guardaré la contraseña del usuario en el payload porque el token está firmado». ¿Qué le responderías?
4. ¿En qué casos la API responde 401 y en cuáles 403? Da un ejemplo de tu laboratorio para cada caso.
5. ¿Por qué el filtro JWT se registra «antes» de `UsernamePasswordAuthenticationFilter`?

Rúbrica de evaluación (20 puntos)

Criterio	Excelente	Logrado	En proceso	Puntos
Entorno y proyecto (E1, E2)	Proyecto en JDK 21 con dependencias correctas y ejecución sin errores	Funciona con pequeñas correcciones	No logra ejecutar	3
JwtService y prueba JUnit (Ejemplo 1, Ejercicio 1)	Genera y valida tokens; 2 pruebas en verde; tabla de la Parte A correcta	Una prueba falla o la tabla tiene errores	No genera o no valida tokens	4
Login (Ejemplo 2, Ejercicio 2)	200, 400 y 401 correctos con mensajes claros	Falta un caso de error	Solo el caso exitoso	4
Filtro y reglas por rol (Ejemplo 3, Ejercicio 3)	Las 14 pruebas correctas; mis-notas solo muestra datos propios	Entre 10 y 13 pruebas correctas	Menos de 10 pruebas correctas	5
Evidencias	Todas las capturas, ordenadas y legibles	Falta alguna captura	Evidencias incompletas	2
Buenas prácticas	Clave por variable de entorno, orden de reglas y código ordenado	Cumple parcialmente	No aplica buenas prácticas	2

¡Buen trabajo! Practica variaciones: cambia el rol, altera un token a propósito y observa cómo reacciona la firma. Comparte tus dudas en el foro del curso: ayudan a todo el grupo.